

Modern Storages and Data Warehousing Week 8 - Airflow

Попов Илья, i.popov@hse.ru

Ресар прошлых занятий

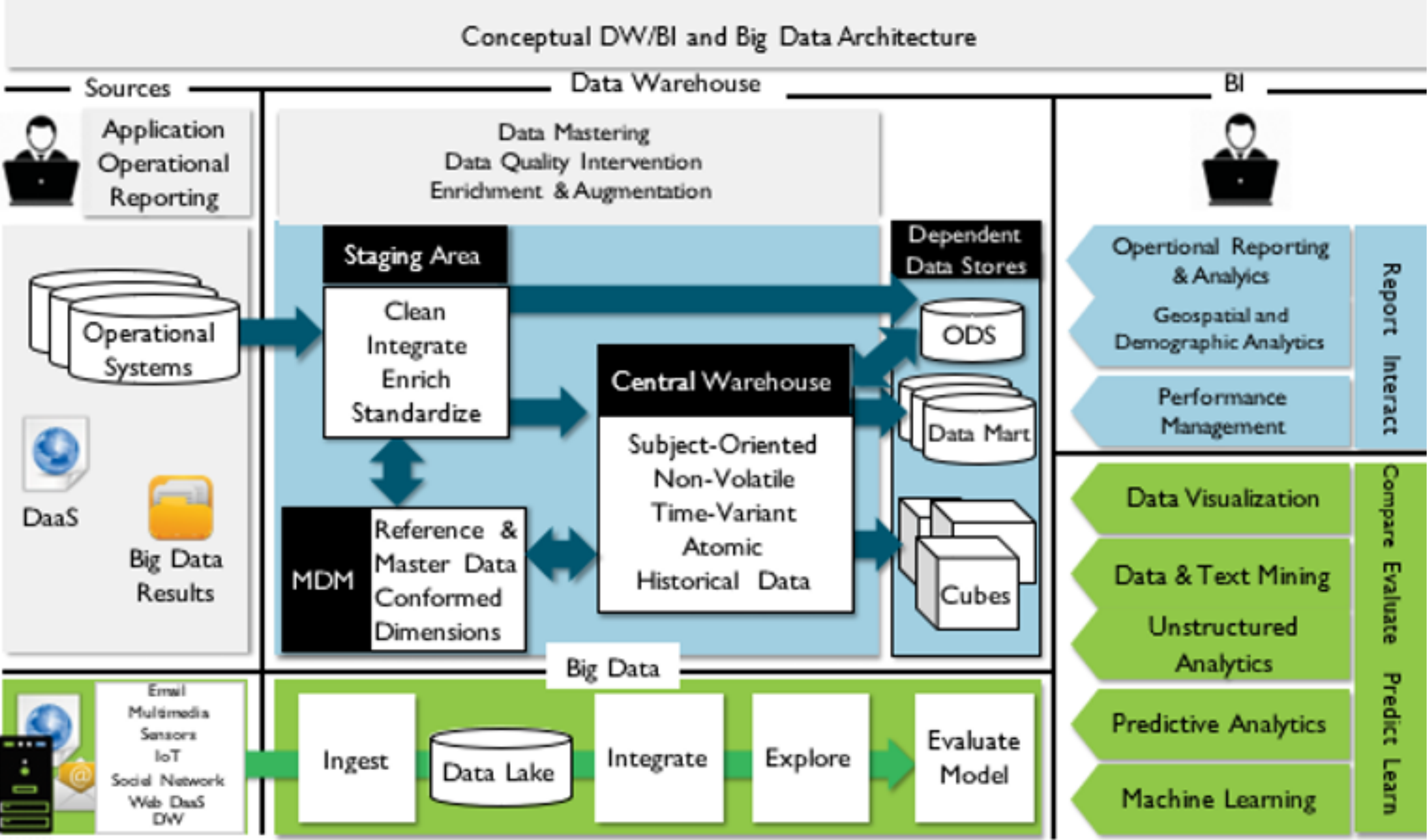


Figure 5: Data Warehouse Concept

1 - Оркестрация

Мотивация

- › Мы написали процесс, который везет изменения из источников к нам в целевую витрину
- › Хотим сделать так, чтобы мы его не запускали руками, а он включался автоматически по какому-нибудь условию или расписанию

Crontab

- › Системный daemon Linux
- › Работает из коробки
- › Может выполнять произвольную bash-команду по заданному расписанию

Crontab

“At 22:00 on every day-of-week from Monday through Friday.”

next at 2023-10-20 22:00:00 random

022* *1-5

minute
(month)

hour

day
(month)

month

day
(week)

*	any value
'	value list separator
-	range of values
/	step values
@yearly	(non-standard)
@annually	(non-standard)
@monthly	(non-standard)
@weekly	(non-standard)
@daily	(non-standard)
@hourly	(non-standard)
@reboot	(non-standard)

Crontab

Как с ним работать:

- › Команда `crontab` -е открывает файл с командами для cron (default `/etc/crontab`)
- › Каждая команда на новой строке, команда имеет вид:
`* * * * * /path/to/exec`

Crontab

Плюсы:

› Простой и безотказный, как автомат Калашникова

Минусы:

› Как и у автомата Калашникова, у него только одно простое применение, и больше он ничего не умеет

Что мы можем еще сделать?

- › Запустить что-то в CI (GitHub CI, GitLab CI, Jenkins)
- › Использовать проприетарный GUI-based ETL
 - › MS SQL SERVER INTEGRATION SYSTEM
 - › ORACLE DATA INTEGRATOR
 - › INFORMatica POWER CENTER
 - › SAS DATA INTEGRATION STUDIO
- › Написать что-то свое с нуля (обычно - обертка над Crontab, но есть Яндекс с его Nirvana / Reactor)

Вселенная opensource code-based ETL фреймворков



Airbnb



Apache Airflow

- › Написали в 2014 внутри Airbnb
- › 2016 - выпустили в OpenSource в рамках Apache Incubator
- › 2019 - top-level проект Apache
- › 2020 - Apache Airflow 2.0



Apache Airflow

- › Написали в 2014 внутри Airbnb
- › 2016 - выпустили в OpenSource в рамках Apache Incubator
- › 2019 - top-level проект Apache
- › 2020 - Apache Airflow 2.0
- › **Сентябрь 2024** - релиз спецификаций к Apache Airflow 3.0



Почему именно он?

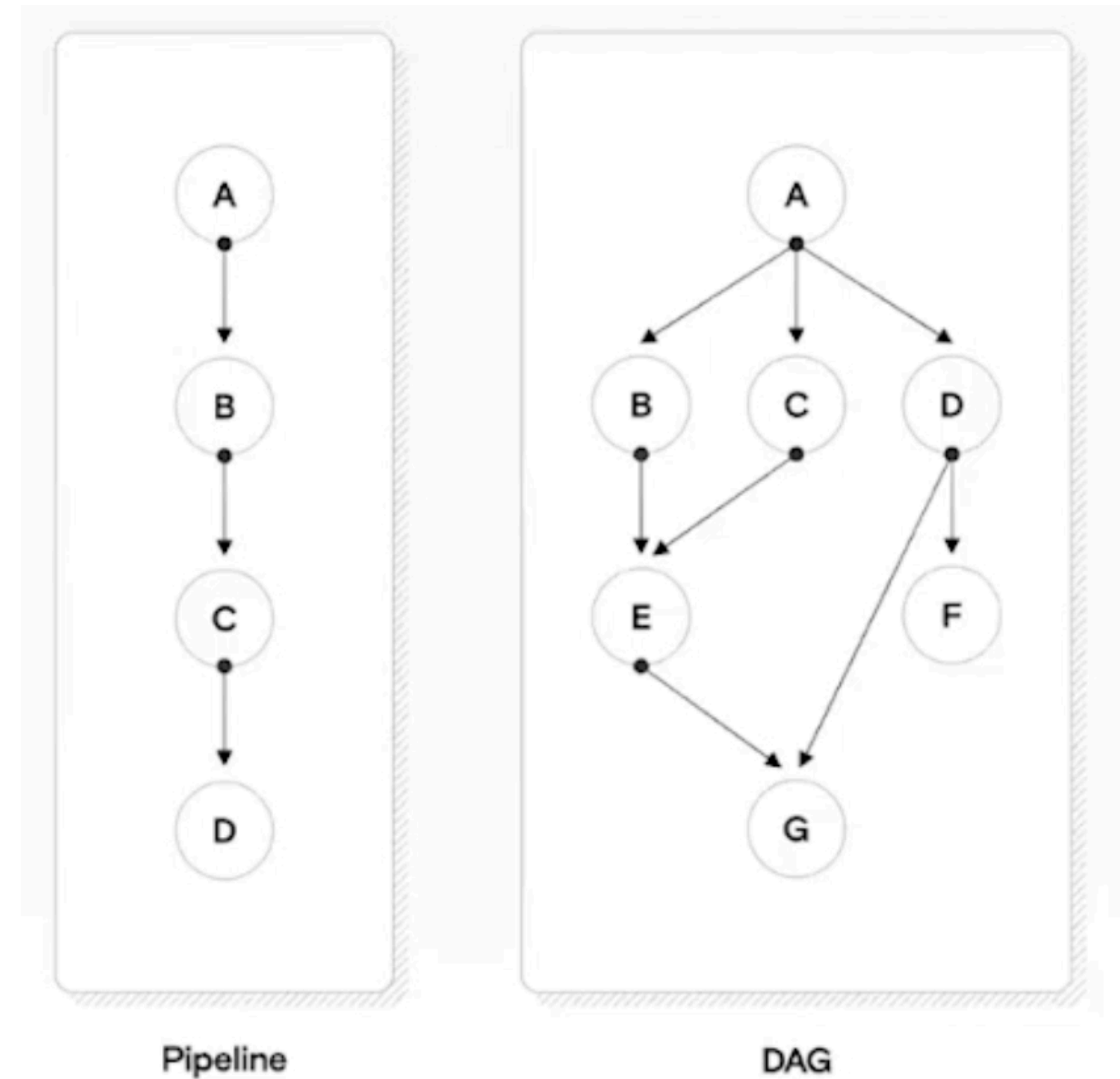
- › Open source
- › Великолепная документация (!)
- › Простой код на Python
- › Отличный Web UI
- › Встроенные алерты и мониторинги
- › Легко масштабируется
- › Легко кастомизируется и дорабатывается
- › Огромное комьюнити



DAG

DAG (Directed Acyclic Graph) -
направленный ациклический
граф.

DAG объединяет отдельные
задачи в единый Pipeline, в
котором явно прослеживаются
зависимости узлов друг от друга.



Виды операторов Airflow

Сенсор - внутри имеет произвольную функцию. Она выполняется в течение указанного времени, при падении/логическом “нет” запускается повторно через какое-то время. Как только эта функция возвращает логическое “да” - продолжает выполнение графа.

Пример: `SqlSensor`, `PythonSensor` и т.д.

Оператор - выполняет произвольное действие. Пример: `SqlOperator`, `PythonOperator` и т.д.

2 - Шаблонизация

Мотивация

- › Иногда мы хотим, чтобы наш код генерировался динамически
- › Для этого мы хотим написать шаблон, и потом подставлять в него куски кода в зависимости от каких-то условий / переменных среды

Jinja

- › Выпущен в 2008 году
- › Сразу писалась командой Pallets Project как opensource проект
- › Написан на Python
- › Похож на стандартный шаблонизатор django



Обращение к переменным и функциям

```
>>> Template("{{ var }}").render(var=12)
'12'
>>> Template("{{ var }}").render(var="hello")
'hello'
>>>
```

```
>>> def foo():
...     return "foo() called"
...
>>>
>>> Template("{{ foo() }}").render(foo=foo)
'foo() called'
>>>
```


Циклы и условия

```
{% if user %}
    {% if user.newbie %}
        <p>Display newbie stages</p>
    {% elif user.pro %}
        <p>Display pro stages</p>
    {% elif user.ninja %}
        <p>Display ninja stages</p>
    {% else %}
        <p>You have completed all states</p>
    {% endif %}
{% else %}
    <p>User is not defined</p>
{% endif %}
```

```
{% set user_list = ['tom', 'jerry', 'spike'] %}

<ul>
{% for user in user_list %}
    <li>{{ user }}</li>
{% endfor %}
</ul>
```

А еще...

- › Можно писать свои функции - макросы
- › Можно наследовать и вкладывать шаблоны друг в друга
- › Можно применять фильтры и модифицировать множества до рендера

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}Default Title{% endblock %}</title>
</head>
<body>

  {% block nav %}
    <ul>
      <li><a href="/home">Home</a></li>
      <li><a href="/api">API</a></li>
    </ul>
  {% endblock %}

  {% block content %}

  {% endblock %}
</body>
</html>
```

```
{% extends 'base.html' %}

{% block content %}
  {% for bookmark in bookmarks %}
    <p>{{ bookmark.title }}</p>
  {% endfor %}
{% endblock %}
```

А зачем?

- › В основном - нужно web-dev'ам
- › Но Jinja де-факто стала стандартом в кодогенерации
- › Это знание нам нужно для того, чтобы понимать генерацию в Airflow и dbt

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}Default Title{% endblock %}</title>
</head>
<body>

  {% block nav %}
    <ul>
      <li><a href="/home">Home</a></li>
      <li><a href="/api">API</a></li>
    </ul>
  {% endblock %}

  {% block content %}

  {% endblock %}
</body>
</html>
```

```
{% extends 'base.html' %}

{% block content %}
  {% for bookmark in bookmarks %}
    <p>{{ bookmark.title }}</p>
  {% endfor %}
{% endblock %}
```


3 - Продвинутые сценарии

Airflow

(live demo)